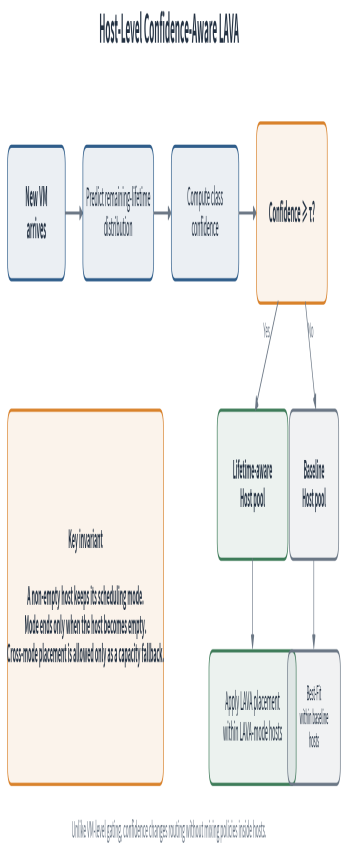
Revisiting Lifetime-Aware VM Allocation: Prediction Sensitivity and Host-Level Confidence-Aware Scheduling

Course Project Author

Course Project Report · Frozen implementation: commit 377cbb1 · October 2026



Overview of the proposed host-level confidence-aware scheduling mechanism.

Abstract

Virtual-machine placement is a multidimensional bin-packing problem with incomplete information: two VMs that fit equally well today may leave very different fragmentation after one exits. LAVA, published at MLSys 2025, uses learned lifetime distributions, repeated prediction, and host lifecycle states to improve the number of empty hosts. This project independently reimplements the public scheduling ideas in an event-driven simulator and studies a question that is difficult to isolate in production: how does lifetime-prediction quality affect scheduling benefit? Across paired synthetic traces, LAVA shows statistically significant gains only when controlled prediction accuracy reaches 85%; an oracle confirms substantial headroom. We then evaluate VM-Level Confidence-Aware LAVA, which falls back to Best-Fit for low-confidence arrivals. That intuitive design does not significantly improve LAVA because per-VM switching mixes incompatible policies within the same host. Guided by this negative result, we propose Host-Level Confidence-Aware LAVA, which assigns each non-empty host a stable scheduling mode. On held-out seeds, the host-level variant improves empty-host percentage by 0.241 points over Best-Fit, 0.285 points over LAVA, and 0.190 points over the VM-level variant under the default synthetic predictor; all three improvements are statistically significant. The study shows that uncertainty information becomes useful only when the control policy preserves the state granularity of the scheduling mechanism.

Keywords: VM scheduling, bin packing, lifetime prediction, uncertainty, confidence-aware scheduling, data-center efficiency

1. Introduction

Cloud schedulers continuously assign virtual machines (VMs) to physical hosts. Although a placement may satisfy CPU and memory constraints, it can still create stranded capacity: free CPU without enough memory, or free memory without enough CPU, that cannot serve the next request. Fragmentation also reduces the number of completely empty hosts, which matters for energy management, large-VM obtainability, maintenance, and failure-domain flexibility.

Conventional Best-Fit scheduling optimizes current resource packing but ignores when allocations will disappear. Lifetime-aware scheduling adds a temporal dimension: VMs expected to leave at similar times should share hosts so that entire hosts become empty together. LAVA extends earlier lifetime alignment by predicting distributions rather than a single fixed label, refreshing predictions as uptime grows, and adapting host state when predictions are wrong [1]. The original system is integrated with Google Borg and evaluated using production traces and deployments, resources that are not public.

This project therefore targets **mechanistic reproduction**, not numerical replication of Google's production results. We implement the public algorithms in a transparent synthetic simulator, preserve the paper's controlled-noise protocol, and explicitly separate paper claims from our own findings. The work asks two research questions:

- **RQ1:** How strongly does LAVA's scheduling benefit depend on lifetime-prediction quality?
- **RQ2:** Can confidence-aware fallback improve robustness when individual lifetime predictions are unreliable?

The first question yields a clear sensitivity curve and an observed significance threshold. The second produces both a useful negative result and a successful refinement: VM-level fallback is not enough, but host-level mode isolation significantly improves the default synthetic-model regime.

Contributions

- An event-driven, two-resource implementation of Best-Fit, LA-Binary, NILAS, LAVA, VM-Level CA-LAVA, and Host-Level CA-LAVA.
- A paired multi-seed evaluation with time-weighted metrics, Student-t 95% confidence intervals, oracle controls, and resource-safety checks.
- A leakage-free analysis of prediction accuracy and class-probability confidence.
- A failure-driven algorithmic contribution showing that host-level policy consistency—not per-request gating alone—is necessary to exploit confidence effectively.

2. Problem Studied

Each host has normalized CPU and memory capacity of one. A VM request has an arrival time, duration, CPU demand, and memory demand. The online scheduler must immediately choose a feasible host without knowing the VM's realized lifetime. The objective is not only to admit every VM but also to maximize the time-weighted fraction of empty hosts while maintaining dense packing among non-empty hosts.

The temporal difficulty is cumulative. If many VMs predicted to be short-lived share a host and even one is actually long-lived, that residual VM can prevent the host from becoming empty. LAVA addresses this by repeatedly estimating remaining lifetime, grouping VMs into lifetime classes, and recycling hosts whose original cohort has mostly departed. Our central concern is how these stateful rules behave under calibrated but uncertain predictions.

3. Methodology

3.1 Baselines and reproduced algorithms

Method	Placement signal	Prediction behavior	Role in this study
Best-Fit	Residual CPU and memory after placement	None	Resource-only baseline

Method	Placement signal	Prediction behavior	Role in this study
LA-Binary	Short/long lifetime class, then Best-Fit	One prediction at creation	Earlier lifetime-alignment baseline
NILAS	Temporal extension cost, then Best-Fit	Remaining lifetime is re-predicted	Non-invasive lifetime score
LAVA	Host lifecycle class and open/recycling state	Distribution-based re-prediction	Primary reproduced method

Best-Fit minimizes a two-dimensional residual-capacity score. LA-Binary separates VMs at a two-hour threshold but keeps the initial label fixed. NILAS estimates how much a new VM would extend the latest predicted host exit and discretizes this temporal cost before applying Best-Fit. LAVA maintains *empty*, *open*, and *recycling* states. An open host enters recycling when CPU or memory reaches 90%; residual VMs and deadlines then cause lifetime-class decreases or increases, adapting to under- and overprediction.

3.2 Lifetime distributions and confidence

The open reproduction uses profile-specific two-exponential mixtures. For uptime u, the predictor computes the conditional expectation E[T-u | T>u]. Confidence is the probability that the remaining lifetime falls inside the predicted lifetime-class interval. For controlled-accuracy experiments, we follow Appendix G.1 of LAVA: each VM is assigned to a correct or incorrect bucket; Gaussian error is applied in log10 space with sigma 0.001 or 3; predictions are capped at 14 days. The scheduler receives only the noisy prediction and the globally calibrated error model—never the realized future lifetime or hidden bucket label.

3.3 Confidence-aware variants

VM-Level CA-LAVA applies LAVA when confidence exceeds a threshold and otherwise applies Best-Fit. This design is intuitive but permits consecutive arrivals to apply different policies to the same non-empty host.

Host-Level CA-LAVA preserves the same confidence decision but routes high-confidence arrivals to lifetime-aware hosts and low-confidence arrivals to baseline hosts. Once a non-empty host acquires a mode, it retains that mode until empty. Cross-mode placement is allowed only if no same-mode or empty host can admit the VM; this capacity fallback preserves admission fairness.

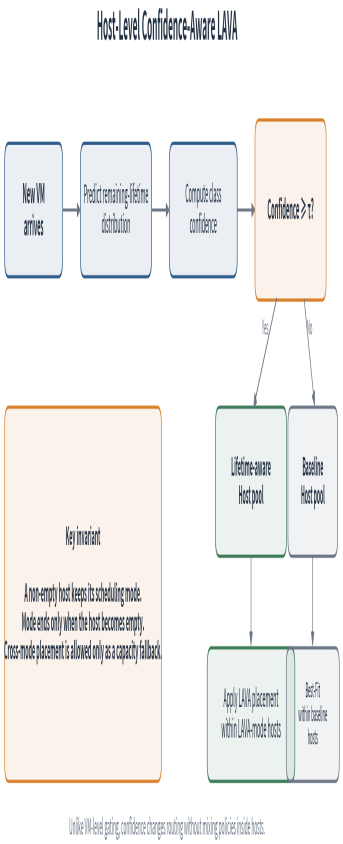


Figure 1. Host-Level CA-LAVA. The key invariant is stable scheduling mode for each non-empty host.

4. Experimental Setup



Figure 2. Leakage-free validation and held-out testing protocol.

4.1 Simulator and workload

The event-driven simulator models 180 homogeneous hosts and two normalized resource dimensions. Each main trace contains 2,500 VMs with an average arrival rate of eight VMs per simulated hour. VM profiles—interactive, batch, and service—use different two-exponential lifetime mixtures; VM shapes include both balanced and CPU/memory-skewed requests. Metrics are integrated over time from the first arrival through the final VM exit rather than sampled only at arrival events.

4.2 Metrics and statistical method

- Primary metric: mean time-weighted empty-host percentage; reported differences are percentage points (pp) relative to the same-seed Best-Fit run.
- Secondary checks: non-empty-host packing density, scheduling failures, accepted VM count, and maximum CPU/memory occupancy.
- Uncertainty: paired differences across 20 seeds with two-sided Student-t 95% confidence intervals.
- Fairness: every policy receives identical traces; all reported main experiments have zero scheduling failures, identical accepted-VM counts, and no oversubscription.

4.3 Validation and test splits

The initial accuracy sweep and VM-level study use test seeds 0–19. The final host-level study deliberately uses a new split: validation seeds 200–205 choose the threshold from {0.1, 0.2, 0.3, 0.5, 0.7, 0.8, 0.9}, while held-out seeds 20–39 provide all final significance claims. Threshold 0.3 is selected before examining the final test statistics.

4.4 Reproducibility boundary

The project does not claim to reproduce Google's absolute production gains. Borg source code, production traces, feature pipelines, higher-priority scheduler scores, warm-up behavior, and deployment controls are unavailable. The reproducible claim is narrower: public algorithmic mechanisms, controlled prediction errors, paired synthetic workloads, and a documented extension are implemented and tested end to end.

5. Reproduction Results

5.1 Prediction quality controls LAVA benefit

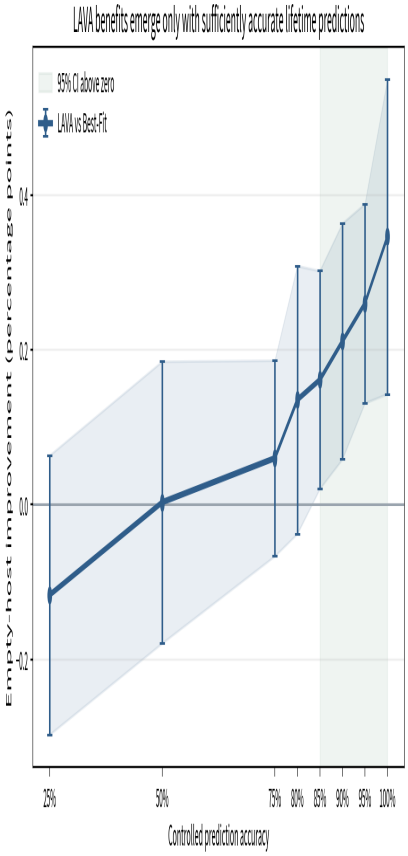


Figure 3. LAVA improvement versus controlled prediction accuracy. Error bars show paired 95% confidence intervals.

At 25%–80% accuracy, the confidence interval includes zero. The first tested point with a statistically significant positive improvement is 85%: +0.161 pp with 95% CI [0.021, 0.302] in the original split. Gains grow at 90%, 95%, and 100%. Under the default synthetic predictor, LAVA is -0.026 pp with CI [-0.168, 0.116], so the small negative mean is not evidence of degradation. A strict oracle reaches +0.721 pp [0.548, 0.894], demonstrating substantial algorithmic headroom.

These values should not be compared numerically with the original paper's Google-trace simulation or production deployment. Their significance is internal: the same simulator, workload family, baseline, and paired seeds reveal a monotonic high-accuracy regime and a clear gap between the default predictor and oracle.

5.2 VM-Level CA-LAVA: an informative negative result

The VM-level gate does not significantly outperform LAVA. At 25% controlled accuracy it shows a small positive trend, but the paired interval crosses zero. At 50% and above, threshold 0.3 admits nearly every request to LAVA, so the two algorithms are identical. Under the default synthetic model, the gate uses LAVA for only 14.9% of arrivals yet still fails to improve significantly. This result indicates that confidence-aware selection cannot be evaluated as a stateless per-request switch when the underlying scheduling algorithm manages host state.

6. Failure Analysis and Host-Level Improvement

6.1 Diagnosing host-mode contamination

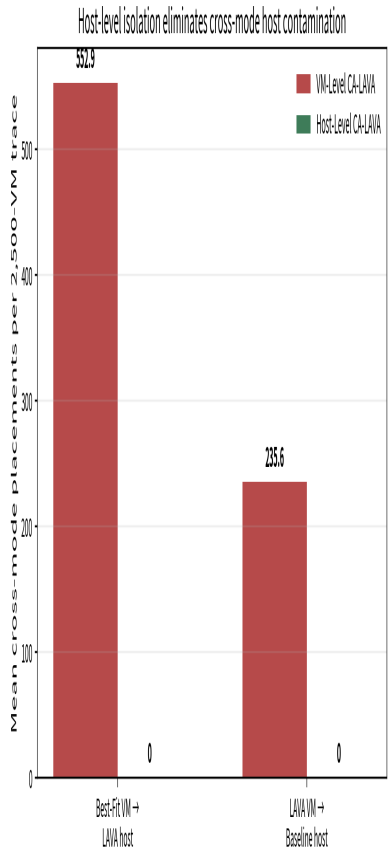


Figure 4. Mean cross-mode placements under the default synthetic predictor on held-out test seeds.

The default synthetic model produces 252.58 gate changes per 1,000 arrivals. VM-Level CA-LAVA consequently places an average of 552.85 Best-Fit decisions into LAVA-mode hosts and 235.55 LAVA decisions into baseline-mode hosts per 2,500-VM trace. The host-level variant sees the same confidence sequence but reduces both contamination counts to zero. This controlled contrast isolates mode mixing as the mechanism changed by the new design.

6.2 Threshold ablation

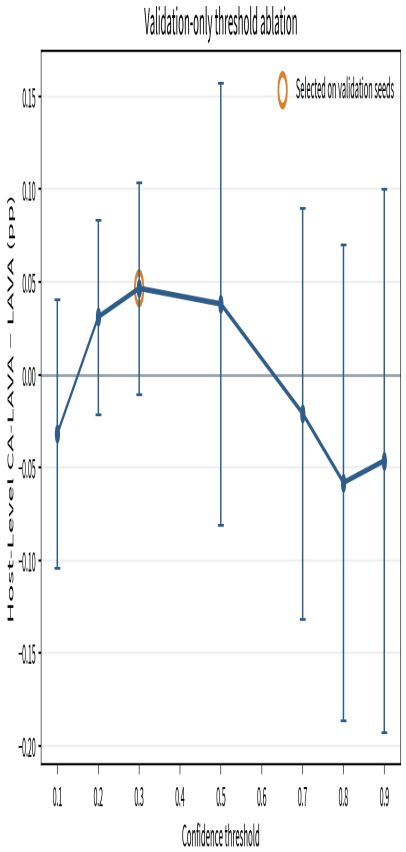


Figure 5. Host-level threshold selection on validation seeds only. The selected threshold is 0.3.

The validation curve is noisy, as expected from six seeds and heterogeneous prediction regimes. Threshold 0.3 has the best average host-level improvement over LAVA and VM-Level CA-LAVA on the validation set. It is then frozen before the held-out test. We do not retune the threshold after observing the final results.

7. Final Results

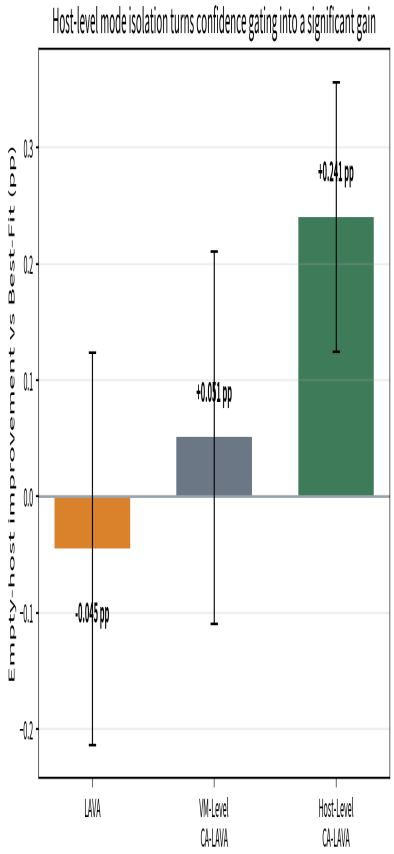


Figure 6. Default synthetic predictor on new held-out seeds 20–39. Error bars are paired 95% confidence intervals.

Under the default synthetic predictor, Host-Level CA-LAVA improves empty-host percentage by +0.241 pp [0.125, 0.357] over Best-Fit. It also improves by +0.285 pp [0.112, 0.459] over LAVA and +0.190 pp [0.004, 0.375] over VM-Level CA-LAVA. All three intervals exclude zero. This is the principal algorithmic result of the project.

Table 1. Final held-out results (mean empty-host improvement versus Best-Fit, pp; 95% CI).

Prediction setting	LAVA	VM-Level CA-LAVA	Host-Level CA-LAVA
Controlled 25%	-0.005 [-0.131, 0.122]	-0.027 [-0.157, 0.103]	+0.023 [-0.105, 0.151]
Controlled 50%	-0.012 [-0.173, 0.148]	-0.012 [-0.173, 0.148]	-0.012 [-0.173, 0.148]
Controlled 75%	+0.070 [-0.074, 0.214]	+0.070 [-0.074, 0.214]	+0.070 [-0.074, 0.214]
Controlled 85%	+0.200 [0.040, 0.359]	+0.200 [0.040, 0.359]	+0.200 [0.040, 0.359]
Controlled 100%	+0.406 [0.256, 0.557]	+0.406 [0.256, 0.557]	+0.406 [0.256, 0.557]
Synthetic model	-0.045 [-0.213, 0.124]	+0.051 [-0.109, 0.211]	+0.241 [0.125, 0.357]
Oracle	+0.584 [0.463, 0.705]	+0.584 [0.463, 0.705]	+0.584 [0.463, 0.705]

At controlled accuracy of 50% and above, all arrivals pass the selected threshold, making both confidence-aware variants equivalent to LAVA. Thus, the host-level extension preserves all high-accuracy and oracle gains rather than trading them for robustness. At 25%, it has a positive but non-significant trend. Its distinct benefit appears in the realistic default synthetic model, where class probabilities vary by profile and create frequent high/low routing decisions.

8. Findings and Discussion

Finding 1: prediction quality is a first-order scheduling variable.

The oracle and controlled-accuracy curves show that LAVA's mechanism has useful headroom, but prediction error can erase the gain. Reporting a single model accuracy is therefore insufficient; scheduler evaluations should expose a sensitivity curve and uncertainty intervals.

Finding 2: uncertainty does not automatically yield better decisions.

VM-Level CA-LAVA uses a valid, leakage-free confidence estimate, yet does not significantly improve the scheduler. Confidence is only valuable when the intervention respects the temporal and stateful structure of the system.

Finding 3: policy granularity should match state granularity.

LAVA's adaptation occurs at host level through open/recycling states and lifetime classes. Switching the scoring policy independently for every VM mixes incompatible objectives inside those hosts. Stable host modes align the decision boundary with the state boundary and transform the same confidence sequence into a significant gain.

Finding 4: negative results can identify the right mechanism.

The host-level proposal was not chosen by searching workloads for a positive result. It follows directly from instrumented failure evidence: hundreds of cross-mode placements in the VM-level design. The final test then uses new seeds and a threshold fixed on independent validation seeds. This sequence—negative result, diagnosis, mechanism-driven redesign, held-out evaluation—is central to the credibility of the improvement.

9. Limitations and Threats to Validity

- **Synthetic workload.** Three profile-level mixtures cannot represent the feature diversity, non-stationarity, restarts, resizing, or correlations in production traces.
- **Simplified predictor.** The default model is an analytical mixture rather than the original boosted-tree distribution model. Confidence has coarse profile-level resolution.
- **Simplified scheduler.** The reproduction omits Borg's hierarchical architecture, higher-priority business constraints, warm-up deployment behavior, and dynamic resource management.
- **Scale.** Each main trace contains 2,500 VMs and 180 hosts. Twenty paired seeds quantify stochastic uncertainty but do not substitute for multi-pool production evaluation.
- **Objective scope.** Empty-host percentage is the principal metric. Energy, migration cost, maintenance velocity, and availability are discussed conceptually but not modeled directly.

These limitations restrict external validity, not the internal paired comparisons. Every reported policy sees the same traces and admission constraints, and the host-level result is confirmed on previously unused seeds.

10. Conclusion

This project reproduces the core public ideas of lifetime-aware VM allocation and shows that their benefit depends strongly on prediction quality. In the synthetic environment, 85% is the first tested controlled-accuracy level with statistically significant positive LAVA improvement; oracle predictions expose larger headroom. A natural VM-level confidence gate does not significantly improve LAVA because request-level policy switching contaminates host-level lifecycle structure. Host-Level Confidence-Aware LAVA resolves that mismatch by keeping each non-empty host in a stable mode. On held-out seeds under the default synthetic predictor, it significantly improves over Best-Fit, LAVA, and VM-Level CA-LAVA while preserving LAVA's high-accuracy and oracle gains.

The broader lesson is that prediction uncertainty becomes actionable only when the control policy respects the state granularity of the system. Future work should evaluate host-level isolation using richer calibrated predictors and real multi-pool traces, without changing the frozen claims of this course project.

References

[1] J. Ling et al. “LAVA: Lifetime-Aware VM Allocation with Learned Distributions and Adaptation to Mispredictions.” *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2412.09840.

[2] A. Verma et al. “Large-scale cluster management at Google with Borg.” *Proceedings of EuroSys*, 2015.

[3] L. Barbalho et al. Lifetime-alignment work for improving VM bin packing, 2023, as discussed and evaluated against in LAVA [1].

Artifact

The frozen implementation, tests, raw per-seed results, and analysis scripts are recorded at Git commit **377cbb1** and tag **experiment-final-v2**. The project contains 12 passing unit tests and explicit checks for admission equality and resource oversubscription.